

Progetto di esame

Classificazione dello stato fetale

Studente: Claudio Scamporlino

Matricola: [redacted for publication]

Esame: Apprendimento Automatico e Apprendimento Profondo (UTSR)

Docente: Prof.ssa Noemi Scarpato

Progetto: Classificazione dello stato fetale

Dataset: Cardiotocography - UCI ML Repository (ID: 193)

1. Introduzione

Il progetto consiste in un esperimento di classificazione in Python. Ho scelto di occuparmi di un dataset medico che contiene misurazioni estratte da cardiocotogrammi fetali, cioè quei tracciati che si fanno in gravidanza per monitorare il battito del feto e le contrazioni uterine.

Ho scelto questo dataset per diversi motivi. Innanzitutto ha 2.126 campioni e 21 feature numeriche, una dimensione che si presta bene sia agli algoritmi classici sia al deep learning. Il fatto di avere 3 classi (normal, suspect, pathologic) lo rende più interessante di un semplice binario. Infine, trattandosi di medicina, la spiegabilità diventa particolarmente importante: aiuta a capire quali variabili pesano di più nelle decisioni del modello.

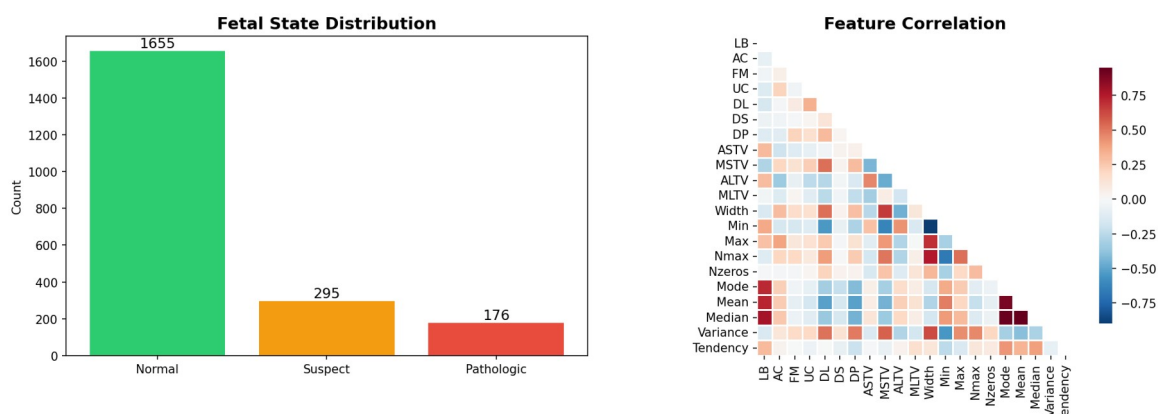
Oltre alla PCA, ai tre classificatori, alle metriche di valutazione e alle visualizzazioni, ho scelto di includere sia l'approccio deep learning sia l'analisi di spiegabilità con SHAP.

2. Il dataset

Il dataset contiene 21 feature misurate automaticamente dai tracciati CTG. Tra queste ci sono: LB (frequenza cardiaca fetale di base), AC (accelerazioni al secondo), FM (movimenti fetali), UC (contrazioni uterine), diverse tipologie di decelerazioni (DL, DS, DP), misure di variabilità a breve e lungo termine (ASTV, MSTV, ALTV, MLTV), e statistiche dell'istogramma della frequenza cardiaca (Width, Min, Max, Mode, Mean, Median, Variance, ecc.).

La variabile target NSP classifica lo stato fetale in tre classi: Normal (1.655 campioni, il 77.8%), Suspect (295, il 13.9%) e Pathologic (176, l'8.3%). Come si vede, il dataset è sbilanciato: la stragrande maggioranza dei casi è normale, il che riflette la realtà clinica ma rende la classificazione delle classi minoritarie più impegnativa.

Un aspetto che ho notato subito guardando la matrice di correlazione è che molte feature sono fortemente correlate tra loro: Mode, Mean e Median, ad esempio, hanno correlazioni superiori a 0.90. Questo ci anticipa che la PCA sarà molto efficace nel ridurre la dimensionalità.



3. Preprocessing

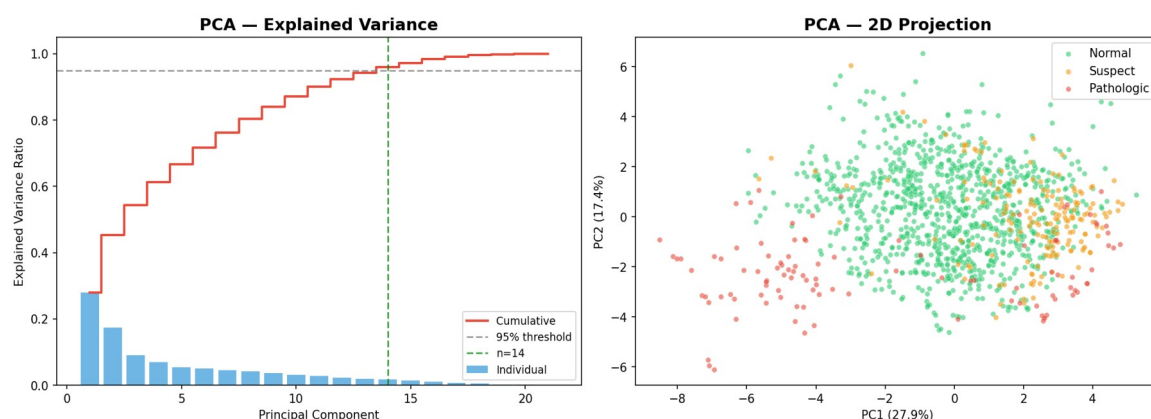
Ho diviso i dati in tre set: training (70%), validation (20%) e test (10%), usando la stratificazione per mantenere le proporzioni delle classi in ciascun set. Ho usato la stessa suddivisione per tutti gli esperimenti (shallow, deep learning e SHAP) così i risultati sono direttamente confrontabili.

Ho applicato lo StandardScaler, che normalizza ogni feature sottraendo la media e dividendo per la deviazione standard. Questo passaggio è fondamentale per la PCA (che altrimenti sarebbe influenzata dalle feature con valori più grandi) e per gli algoritmi sensibili alla scala come SVM e Logistic Regression. Il fit dello scaler l'ho fatto sul training set e poi ho trasformato validation e test con gli stessi parametri.

4. Task 1 - Analisi PCA

Ho applicato la PCA alle feature standardizzate del training set. Analizzando la varianza, ho visto che bastano 14 componenti per conservare il 95% dell'informazione originale. Questa scelta riduce la dimensione del problema eliminando le ridondanze dovute alle forti correlazioni tra le 21 feature di partenza.

Ho anche fatto una proiezione a 2 componenti per visualizzare i dati. Lo scatter plot mostra che la classe Pathologic tende a formare un cluster parzialmente separato, mentre Normal e Suspect hanno vaste zone in cui si sovrappongono. Questo ha subito segnalato che distinguere Normal da Pathologic sarebbe stato più semplice rispetto a distinguere Normal da Suspect.



Ho poi usato le 14 componenti PCA come input per tutti i classificatori successivi.

5. Task 2 - Algoritmi di classificazione

Ho implementato tre classificatori, scelti perché rappresentano approcci diversi al problema:

Logistic Regression: L'ho usata come baseline lineare. Nonostante la sua semplicità, funziona discretamente perché la PCA ha già “raddrizzato” i dati lungo le direzioni di massima varianza.

SVM con kernel RBF: Ho scelto il kernel RBF perché introduce non-linearità e permette al modello di tracciare confini decisionali curvi. Ho impostato `probability=True` per poter calcolare le probabilità e poter valutare il modello tramite la curva ROC AUC, che in ambito clinico è essenziale per capire quanto il sistema sia affidabile.

Random Forest: Un ensemble di 200 alberi decisionali con profondità massima 10. L'ho scelto perché è robusto, gestisce bene i dataset sbilanciati e si adatta bene a SHAP (TreeExplainer).

6. Task 3 - Metriche di valutazione

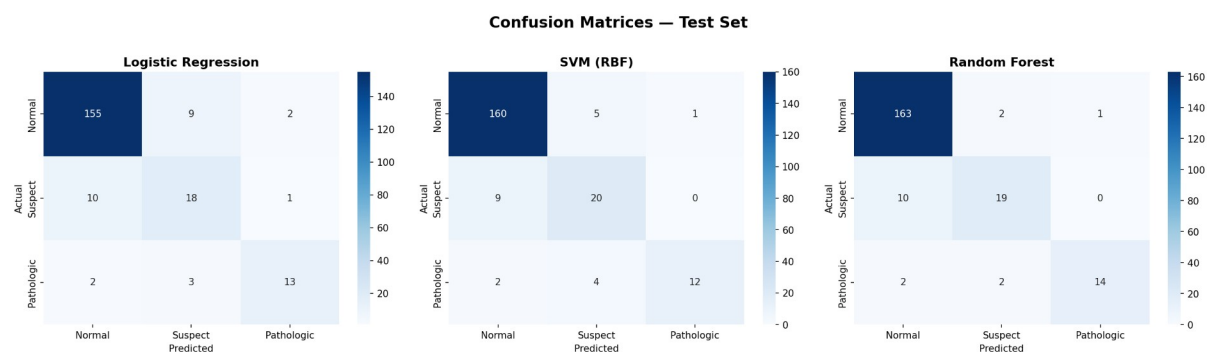
Ho calcolato le metriche sul test set (213 campioni). Precision, Recall e F1-Score sono calcolati con media macro, che dà lo stesso peso a tutte le classi indipendentemente dalla loro frequenza. La ROC AUC è calcolata con strategia One-vs-Rest.

Modello	Accuracy	Precision	Recall	F1-Score	ROC AUC
Logistic Regression	0.873	0.780	0.759	0.769	0.967
SVM (RBF)	0.901	0.850	0.773	0.805	0.968
Random Forest	0.920	0.897	0.805	0.845	0.965
Deep Learning (DNN)	0.911	0.861	0.836	0.848	0.973

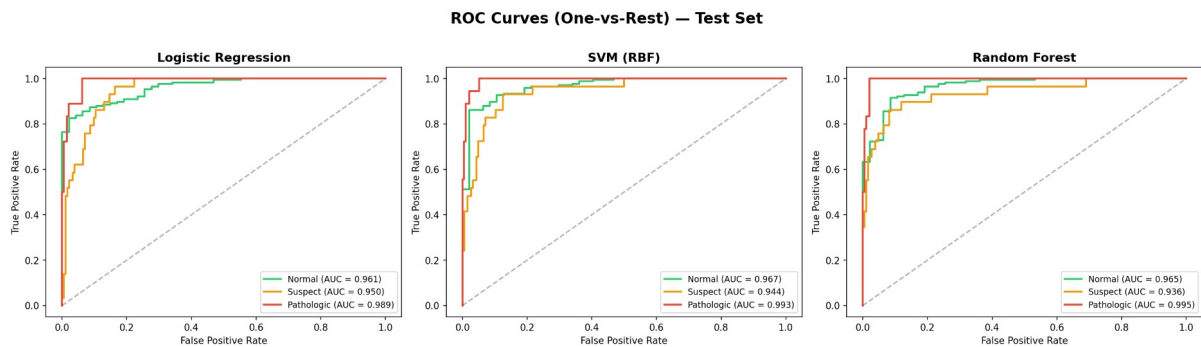
Il Random Forest è il migliore tra i modelli shallow su quasi tutte le metriche. Il DNN mostra un lieve vantaggio numerico in F1-score e ROC AUC, ma il margine rispetto a Random Forest è ridotto e va interpretato con cautela. La SVM ha la precision più alta dopo RF ma un recall più basso: in pratica è più “prudente”, quando predice Pathologic di solito ha ragione, ma ne manca di più tra le classi minoritarie.

7. Task 4 - Confusion Matrix e ROC Curves

Le confusion matrix mostrano un pattern comune a tutti i modelli: i casi Normal vengono classificati molto bene (oltre il 95%), ma la classe Suspect viene spesso confusa con Normal. Questo ha senso clinicamente: i casi sospetti sono per definizione al confine.



Le curve ROC confermano che tutti i modelli hanno eccellenti capacità discriminative, con AUC sopra 0.95 per tutte le classi. La classe Normal è la più facile da identificare, la Pathologic ha AUC leggermente inferiore ma comunque molto alto.



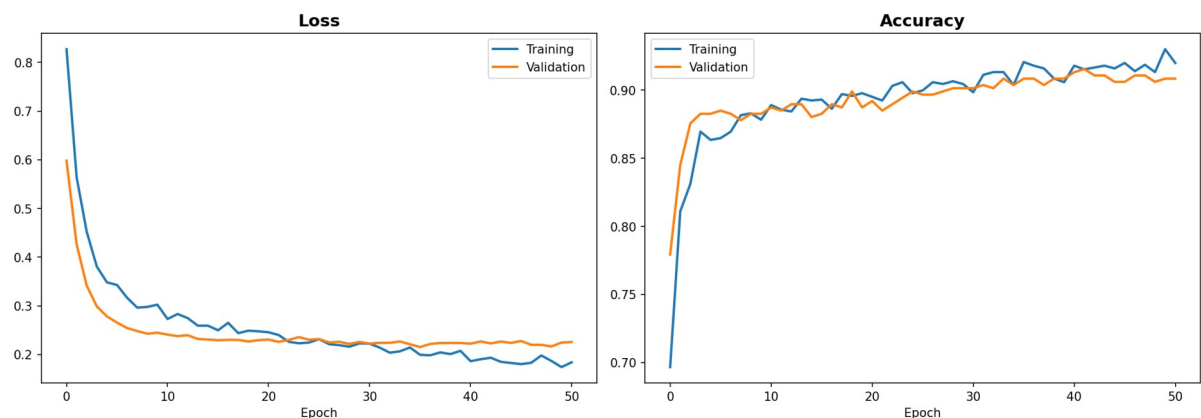
8. Task 5a - Deep Learning

Ho implementato una rete neurale fully connected con TensorFlow/Keras. L'architettura che ho scelto è:

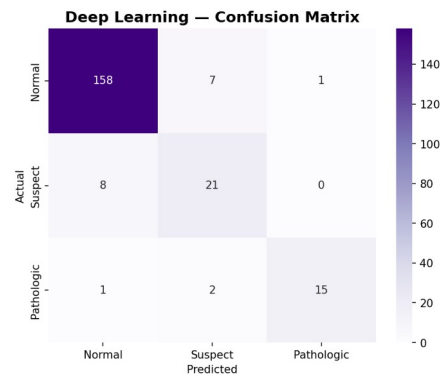
Input(14) → Dense(64, ReLU) → Dropout(0.3) → Dense(32, ReLU) → Dropout(0.2) → Dense(3, softmax)

Ho usato i layer di Dropout per regolarizzare e limitare l'overfitting, dato che il dataset non è molto grande. L'ottimizzatore è Adam con categorical cross-entropy come loss. Ho impostato EarlyStopping sulla validation loss, così l'addestramento si ferma quando la generalizzazione smette di migliorare.

L'uso della PCA è stato mantenuto per coerenza della pipeline e finalità didattiche; non implica ottimalità per tutti i modelli, in particolare per gli ensemble ad alberi.



Sul test set il modello Deep Learning raggiunge Accuracy 0.9108, F1-Score macro 0.8477 e ROC AUC 0.9729: nel confronto sul test set risulta leggermente superiore per F1 macro e ROC AUC, ma le prestazioni restano molto vicine a quelle del Random Forest, il quale resta comunque molto competitivo e più semplice da interpretare con le feature importance.

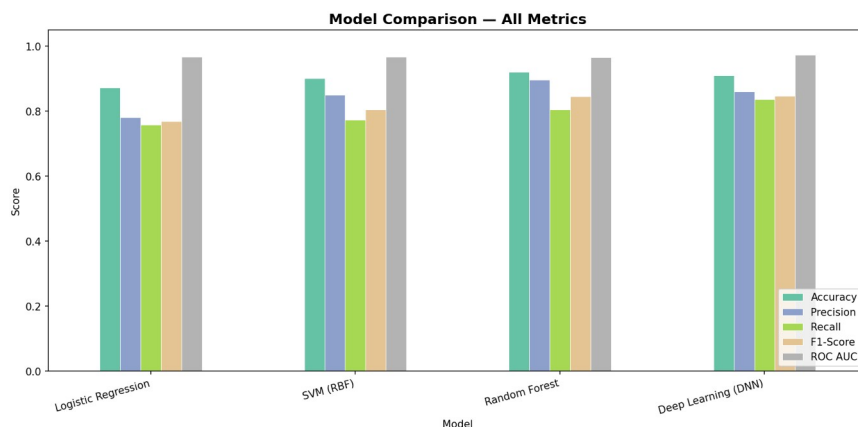


9. Confronto tra i modelli

Guardando la tabella e il grafico di confronto, i quattro modelli hanno prestazioni vicine, ma non identiche. Nel confronto sul test set, Deep Learning (DNN) e Random Forest mostrano prestazioni complessivamente comparabili; il DNN ha un vantaggio numerico lieve su F1 macro e ROC AUC.

Un aspetto comune a tutti i modelli è la difficoltà sulla classe Suspect, spesso confusa con Normal: questo non dipende solo dall'algoritmo scelto, ma dalla sovrapposizione tra profili "normali" e "sospetti" nel dataset.

Questo ci dice una cosa importante: il collo di bottiglia non è l'algoritmo, ma la natura dei dati stessi. Normal e Suspect hanno profili clinici che si sovrappongono, e nessun modello, per quanto sofisticato, riesce a separarli completamente.



10. Task 5b - Spiegabilità

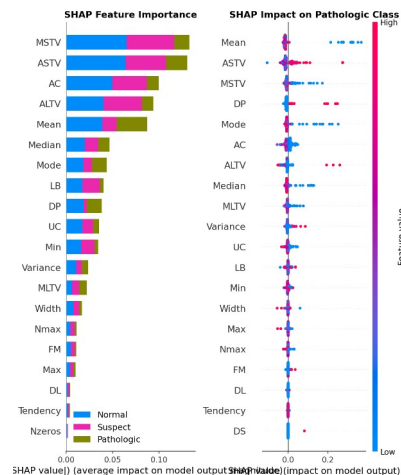
Per la spiegabilità ho usato due livelli, entrambi sul Random Forest addestrato sulle feature originali standardizzate (senza PCA), così le variabili restano interpretabili (es. ASTV, MSTV, AC) e non come componenti principali astratte.

Prima ho analizzato le feature importance del Random Forest (feature_importances_). In questo run le variabili con peso maggiore risultano soprattutto MSTV, ASTV, ALTV, Mean e

Median (ordine basato sulle importanze RF).

Poi ho applicato SHAP con TreeExplainer sullo stesso Random Forest. SHAP può dare enfasi leggermente diverse rispetto alle importanze medie, perché spiega il contributo delle variabili in modo più “locale” e può cambiare a seconda della classe predetta.

Complessivamente, variabilità a breve/lungo termine (es. MSTV/ASTV/ALTV) e segnali legati a dinamica della frequenza (es. correlati ad accelerazioni e pattern dell'istogramma) sono tra i fattori più rilevanti emersi dall'analisi. Questo non sostituisce il giudizio clinico, ma aiuta a capire quali variabili il modello sta usando con più peso nelle decisioni.



11. Conclusioni

Questo progetto mi ha permesso di vedere come si costruisce una pipeline di machine learning completa, dal preprocessing all'interpretazione dei risultati. Alcuni aspetti che mi hanno colpito:

- La PCA conferma molta ridondanza tra le feature: da 21 feature originali a 14 componenti per il 95% di varianza, in linea con le correlazioni osservate (es. Mode/Mean/Median).
- Nel confronto sul test set, DNN e Random Forest ottengono risultati molto vicini: il DNN ha un lieve vantaggio in F1 macro (0.848) e ROC AUC (0.973), mentre il Random Forest resta il migliore tra gli shallow e più immediato da interpretare.
- La parte di spiegabilità è utile soprattutto se letta a due livelli: importanze RF (semplici) e SHAP (più dettagliato). Questo rende più chiaro cosa guida le predizioni.
- Tra i miglioramenti futuri prioritari: gestione esplicita dello sbilanciamento (es. class_weight), validazione più robusta (cross-validation) e tuning sistematico degli iperparametri.
- Limitazioni: il test set contiene 213 campioni, di cui 18 Pathologic; per questa classe le metriche possono variare sensibilmente anche con poche predizioni diverse. I risultati vanno quindi letti come indicativi sul singolo split, in assenza di cross-validation.

Appendice — Codice Python

Di seguito il codice completo del notebook Python, suddiviso per celle.

Cell 1

```
# Step 0 - Imports
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, roc_curve, auc,
    classification_report
)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping

import shap

output_dir = 'outputs'
os.makedirs(output_dir, exist_ok=True)

print("Imports completed.")
print(f"TensorFlow version: {tf.__version__}")
print(f"Output directory: {output_dir}")
```

Cell 2

```
# Step 1 - Load dataset
# The CTG dataset from UCI (ID: 193)
df = pd.read_csv('CTG.csv')

feature_names = [
    'LB',          # FHR baseline (beats per minute)
    'AC',          # Accelerations per second
    'FM',          # Fetal movements per second
    'UC',          # Uterine contractions per second
    'DL',          # Light decelerations per second
    'DS',          # Severe decelerations per second
    'DP',          # Prolonged decelerations per second
    'ASTV',        # % time with abnormal short term variability
    'MSTV',        # Mean short term variability
    'ALTV',        # % time with abnormal long term variability
    'MLTV',        # Mean long term variability
    'Width',       # FHR histogram width
    'Min',         # FHR histogram minimum
    'Max',         # FHR histogram maximum
    'Nmax',        # Number of histogram peaks
    'Nzeros',      # Number of histogram zeros
    'Mode',        # Histogram mode
```



```

    'Mean',      # Histogram mean
    'Median',    # Histogram median
    'Variance',  # Histogram variance
    'Tendency'   # Histogram tendency
]

target_col = 'NSP'
target_labels = {1: 'Normal', 2: 'Suspect', 3: 'Pathologic'}
class_labels = ['Normal', 'Suspect', 'Pathologic']

df_clean = df[feature_names + [target_col]].dropna()
X = df_clean[feature_names].values
y = df_clean[target_col].values.astype(int)

print(f"Dataset loaded: {X.shape[0]} samples, {X.shape[1]} features")
print("Target distribution:")
for val, name in target_labels.items():
    count = (y == val).sum()
    print(f"    {name} (class {val}): {count} ({count/len(y)*100:.1f}%)")

```

Cell 3

```

# Exploratory Data Analysis
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Class distribution
class_counts = pd.Series(y).value_counts().sort_index()
colors = ['#2ecc71', '#f39c12', '#e74c3c']
bars = axes[0].bar([target_labels[i] for i in class_counts.index], class_counts.values,
color=colors)
axes[0].set_title('Fetal State Distribution', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Count')
for bar, count in zip(bars, class_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
        str(count), ha='center', fontsize=12)

# Correlation heatmap
df_features = pd.DataFrame(X, columns=feature_names)
corr = df_features.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, cmap='RdBu_r', center=0, ax=axes[1],
        square=True, linewidths=0.5, cbar_kws={'shrink': 0.8})
axes[1].set_title('Feature Correlation Matrix', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{output_dir}/01_eda.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 4

```

# Split data (70/20/10, stratified) and scale features
# 70/20/10 split with stratification
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=(2/9), random_state=42, stratify=y_train_val
)

print(f"Training set:    {X_train.shape[0]} samples ({X_train.shape[0]/len(X)*100:.0f}%)")
print(f"Validation set: {X_val.shape[0]} samples ({X_val.shape[0]/len(X)*100:.0f}%)")
print(f"Test set:        {X_test.shape[0]} samples ({X_test.shape[0]/len(X)*100:.0f}%)")

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

```

```
print("Scaling completed (fit on train only).")
```

Cell 5

```
# Task 1: PCA analysis (variance + 2D projection)
pca_full = PCA(n_components=None)
pca_full.fit(X_train_scaled)
cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
n_95 = np.argmax(cumulative_variance >= 0.95) + 1

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(1, len(pca_full.explained_variance_ratio_) + 1),
            pca_full.explained_variance_ratio_, alpha=0.7, label='Individual',
            color='#3498db')
axes[0].step(range(1, len(cumulative_variance) + 1), cumulative_variance,
            where='mid', label='Cumulative', color='#e74c3c', linewidth=2)
axes[0].axhline(y=0.95, color='gray', linestyle='--', alpha=0.7, label='95% threshold')
axes[0].axvline(x=n_95, color='green', linestyle='--', alpha=0.7, label=f'n={n_95}'
components')
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance Ratio')
axes[0].set_title('PCA - Explained Variance', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=9)

pca_2d = PCA(n_components=2)
X_train_2d = pca_2d.fit_transform(X_train_scaled)
colors_map = {1: '#2ecc71', 2: '#f39c12', 3: '#e74c3c'}
for cls, label in target_labels.items():
    m = y_train == cls
    axes[1].scatter(X_train_2d[m, 0], X_train_2d[m, 1], c=colors_map[cls],
                    label=label, alpha=0.6, s=20, edgecolors='w', linewidth=0.3)
axes[1].set_xlabel(f'PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}%)')
axes[1].set_ylabel(f'PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}%)')
axes[1].set_title('PCA - 2D Projection (Training Set)', fontsize=14, fontweight='bold')
axes[1].legend()

plt.tight_layout()
plt.savefig(f'{output_dir}/02_pca.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"\n95% variance with {n_95} components (out of {X_train_scaled.shape[1]})")
print(f"First 2 PCs explain {cumulative_variance[1]*100:.1f}% of variance")
```

Cell 6

```
# Apply PCA with optimal components
pca_final = PCA(n_components=n_95)
X_train_pca = pca_final.fit_transform(X_train_scaled)
X_val_pca = pca_final.transform(X_val_scaled)
X_test_pca = pca_final.transform(X_test_scaled)
print(f"Dimensionality reduced: {X_train_scaled.shape[1]} → {X_train_pca.shape[1]} features")
```

Cell 7

```
# Task 2: define and train shallow classifiers on PCA features
# We train Logistic Regression, SVM, and Random Forest on the same transformed data.
classifiers = {
    'Logistic Regression': LogisticRegression(max_iter=1000, solver='lbfgs',
random_state=42),
    'SVM (RBF)': SVC(kernel='rbf', probability=True, random_state=42, gamma='auto'),
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
}

classes = sorted(np.unique(y))
y_test_bin = label_binarize(y_test, classes=classes)
```

```

results = {}
metrics_data = []

for name, model in classifiers.items():
    model.fit(X_train_pca, y_train)
    y_pred = model.predict(X_test_pca)
    y_prob = model.predict_proba(X_test_pca)
    results[name] = {'model': model, 'y_pred': y_pred, 'y_prob': y_prob}
    print(f"Model trained: {name}")

```

Cell 8

```

# Task 3: compute test metrics for each shallow classifier
for name, res in results.items():
    y_pred = res['y_pred']
    y_prob = res['y_prob']

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='macro', zero_division=0)
    rec = recall_score(y_test, y_pred, average='macro', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)
    roc = roc_auc_score(y_test_bin, y_prob, multi_class='ovr', average='macro')

    metrics_data.append({
        'Model': name,
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'F1-Score': f1,
        'ROC AUC': roc
    })

    print(f"\n--- {name} ---")
    print(f"Accuracy: {acc:.4f} | Precision: {prec:.4f} | Recall: {rec:.4f} | F1: {f1:.4f} | ROC AUC: {roc:.4f}")
    print("Classification report:")
    print(classification_report(y_test, y_pred, target_names=class_labels, zero_division=0))

metrics_df = pd.DataFrame(metrics_data).set_index('Model')
print("\n" + "=" * 60)
print("Summary metrics (test set)")
print(metrics_df.round(4).to_string())

```

Cell 9

```

# Task 4a: confusion matrices on test set
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for idx, (name, res) in enumerate(results.items()):
    cm = confusion_matrix(y_test, res['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
                xticklabels=class_labels, yticklabels=class_labels)
    axes[idx].set_title(name, fontsize=13, fontweight='bold')
    axes[idx].set_xlabel('Predicted'); axes[idx].set_ylabel('Actual')
plt.suptitle('Confusion Matrices – Test Set', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(f'{output_dir}/03_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 10

```

# Task 4b: ROC curves (one-vs-rest)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
colors_roc = ['#2ecc71', '#f39c12', '#e74c3c']
for idx, (name, res) in enumerate(results.items()):
    for i, cls in enumerate(classes):

```

```

        fpr, tpr, _ = roc_curve(y_test_bin[:, i], res['y_prob'][:, i])
        roc_val = auc(fpr, tpr)
        axes[idx].plot(fpr, tpr, color=colors_roc[i], linewidth=2,
                        label=f'{class_labels[i]} (AUC = {roc_val:.3f})')
        axes[idx].plot([0, 1], [0, 1], 'k--', alpha=0.3)
        axes[idx].set_xlabel('False Positive Rate'); axes[idx].set_ylabel('True Positive Rate')
        axes[idx].set_title(name, fontsize=13, fontweight='bold')
        axes[idx].legend(loc='lower right', fontsize=9)
plt.suptitle('ROC Curves (One-vs-Rest) – Test Set', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(f'{output_dir}/04_roc_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 11

```

# Task 5a: deep learning model (DNN)
# We keep the same PCA inputs and train/val/test split used above.
y_train_dl = to_categorical(y_train - 1, num_classes=3)
y_val_dl = to_categorical(y_val - 1, num_classes=3)
y_test_dl = to_categorical(y_test - 1, num_classes=3)

model_dl = Sequential([
    Input(shape=(X_train_pca.shape[1],)),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(3, activation='softmax')
])
model_dl.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

print("Model Architecture:")
model_dl.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)
history = model_dl.fit(
    X_train_pca, y_train_dl,
    validation_data=(X_val_pca, y_val_dl),
    epochs=150, batch_size=32, callbacks=[early_stop], verbose=1
)
print(f"\nTraining stopped at epoch {len(history.history['loss'])}")

```

Cell 12

```

# Training curves
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(history.history['loss'], label='Training', lw=2)
axes[0].plot(history.history['val_loss'], label='Validation', lw=2)
axes[0].set_title('Loss', fontsize=14, fontweight='bold'); axes[0].set_xlabel('Epoch');
axes[0].legend()
axes[1].plot(history.history['accuracy'], label='Training', lw=2)
axes[1].plot(history.history['val_accuracy'], label='Validation', lw=2)
axes[1].set_title('Accuracy', fontsize=14, fontweight='bold'); axes[1].set_xlabel('Epoch');
axes[1].legend()
plt.tight_layout()
plt.savefig(f'{output_dir}/05_dl_training_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 13

```

# DL Evaluation
y_prob_dl = model_dl.predict(X_test_pca)
y_pred_dl = np.argmax(y_prob_dl, axis=1) + 1

acc_dl = accuracy_score(y_test, y_pred_dl)
prec_dl = precision_score(y_test, y_pred_dl, average='macro', zero_division=0)

```

```

rec_dl = recall_score(y_test, y_pred_dl, average='macro', zero_division=0)
f1_dl = f1_score(y_test, y_pred_dl, average='macro', zero_division=0)
roc_dl = roc_auc_score(y_test_bin, y_prob_dl, multi_class='ovr', average='macro')

print("Deep Learning (DNN) – Test Set Results")
print(f" Accuracy: {acc_dl:.4f} Precision: {prec_dl:.4f}")
print(f" Recall: {rec_dl:.4f} F1-Score: {f1_dl:.4f}")
print(f" ROC AUC: {roc_dl:.4f}")

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_dl), annot=True, fmt='d', cmap='Purples',
              ax=ax, xticklabels=class_labels, yticklabels=class_labels)
ax.set_title('Deep Learning – Confusion Matrix', fontsize=14, fontweight='bold')
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
plt.tight_layout()
plt.savefig(f'{output_dir}/06_dl_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 14

```

# Compare shallow models vs DNN
dl_row = {'Model': 'Deep Learning (DNN)', 'Accuracy': acc_dl, 'Precision': prec_dl,
          'Recall': rec_dl, 'F1-Score': f1_dl, 'ROC AUC': roc_dl}
all_metrics = metrics_data + [dl_row]
comparison_df = pd.DataFrame(all_metrics).set_index('Model')

print("="*70)
print("FINAL COMPARISON – ALL MODELS")
print("="*70)
print(comparison_df.round(4).to_string())
print(f"\nBest by F1-Score: {comparison_df['F1-Score'].idxmax()} ({comparison_df['F1-Score'].max():.4f})")

fig, ax = plt.subplots(figsize=(12, 6))
comparison_df.plot(kind='bar', ax=ax, colormap='Set2', edgecolor='white', linewidth=0.5)
ax.set_title('Model Comparison – All Metrics', fontsize=14, fontweight='bold')
ax.set_ylabel('Score'); ax.set_ylim(0, 1.05)
ax.set_xticklabels(ax.get_xticklabels(), rotation=15, ha='right')
ax.legend(loc='lower right')
plt.tight_layout()
plt.savefig(f'{output_dir}/07_model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cell 15

```

# Task 5b: explainability on original features
rf_original = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
rf_original.fit(X_train_scaled, y_train)

# 1) Random Forest feature importances
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': rf_original.feature_importances_
}).sort_values('Importance', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df.head(10), x='Importance', y='Feature', color='#4c72b0')
plt.title('Random Forest Feature Importances (Top 10)', fontsize=13, fontweight='bold')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.savefig(f'{output_dir}/08_rf_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

print('Top 5 features by Random Forest importance:')
print(importance_df.head(5).to_string(index=False))

```

```

# 2) SHAP analysis
X_test_df = pd.DataFrame(X_test_scaled, columns=feature_names)
explainer = shap.TreeExplainer(rf_original)
shap_values = explainer.shap_values(X_test_scaled)
shap_values = np.array(shap_values)

# Handle SHAP output format: (samples, features, classes)
if shap_values.shape[0] == X_test_scaled.shape[0]:
    sv_list = [shap_values[:, :, i] for i in range(shap_values.shape[2])]
    sv_pathologic = shap_values[:, :, 2]
else:
    sv_list = [shap_values[i] for i in range(3)]
    sv_pathologic = shap_values[2]

print(f"SHAP values computed. Shape: {shap_values.shape}")

```

Cell 16

```

# SHAP summary plots
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

plt.sca(axes[0])
shap.summary_plot(sv_list, X_test_df, plot_type='bar', class_names=class_labels, show=False)
axes[0].set_title('SHAP Feature Importance', fontsize=13, fontweight='bold')

plt.sca(axes[1])
shap.summary_plot(sv_pathologic, X_test_df, show=False)
axes[1].set_title('SHAP Impact on Pathologic Class', fontsize=13, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{output_dir}/09_shap_original_features.png', dpi=150, bbox_inches='tight')
plt.show()

print('SHAP analysis completed.')
print('Compare these results with RF feature_importances_ for consistency.')

```